ITA03-MOBILE COMPUTING

ASSIGNMENT-5

ASSIGNMENT 5 – CO5

CO5 Coverage: Six Characteristics of mobile app - connectivity, convenience, localization, reachability, security and personalization - Android Intents and Services - Synchronization and Replication of Mobile Data - Android & IOS multimedia framework (Core Video framework and Core Media framework) -Peer- to Peer Architecture for mobile application - Performance and Multithreading -Graphics and UI Performance.

ANDROID APPLICATION FEATURES & PERFORMANCE ENGINEERING

Course Outcome – CO5: Create mobile application design using diverse mobile operating system frameworks to ensure performance efficiency, security, usability, and scalability.

Nature: Individual

Duration: One Week

Total Marks: 50

NAME:K.BALAJI

REG.NO:192324180

Assessment Components

PART A – Design, Analysis & Pseudocode / Architectural Reasoning: 25 Marks PART B – Implementation, Instrumentation & Computational Investigation: 25 Marks **Submission Date:13/11/2026**

PURPOSE OF THE ASSIGNMENT

Modern mobile applications must deliver responsive user experiences while operating under severe constraints of battery, memory, network variability and security threats.

This assignment investigates a fundamental engineering question:

How should a mobile application engineer design, implement and evaluate components so that the application remains efficient, secure, usable and

scalable when the number of concurrent users, data volume and device heterogeneity grows?

You are expected to demonstrate the following progression:

Model → Design Intent/Service → Synchronise → Optimise → Measure → Analyse Trade-offs → Approximate / Heuristic → Evaluate

Simply obtaining a working application is not sufficient. Your architectural reasoning, performance instrumentation, experimental evidence and interpretation are all part of the assessment.

TASK A1 – DESIGN A CONSTRAINT-AWARE INTENT & SERVICE ARCHITECTURE (7 Marks)

Title: Android Intents, Services and Secure Background Work

A1.1 – Model the Problem (1 Mark)

Problem Statement:

A healthcare monitoring application collects sensor readings, uploads the readings to a cloud server when internet connectivity is available and sends notifications when critical health events occur.

1. Input

- Sensor events such as heart rate, temperature and blood pressure.
- Network state (online or offline).
- User preferences.
- Battery level and permission status.

2. Output

- Successfully uploaded sensor records.
- Critical health notifications.
- Upload status and error messages.

3. Decisions at Each Stage

Stage	Decision
Data collection	Whether sensor data should be collected
Data storage	Whether the reading is valid
Upload	Whether internet connectivity is available
Security	Whether data is encrypted

Notification	Whether the reading is critical
Scheduling	Whether the task can execute now or must wait

4. Constraints

- Battery level must be sufficient.
- Background execution must respect Android Doze mode.
- Required permissions must be available.
- Sensitive healthcare data must be encrypted.
- Network connectivity must be available for uploading.

5. Goal Condition

The goal is to collect sensor readings, securely upload all valid records and notify users about critical events without unnecessarily consuming battery or exposing private information.

A1.2 – Define the Runtime State (1 Mark)

The runtime state stores all information required to manage background tasks.

State Variable	**Description**
SensorData	Stores collected sensor readings
NetworkStatus	Online or offline
BatteryLevel	Current battery percentage
PermissionStatus	Granted or revoked
EncryptionStatus	Encrypted or not encrypted
TaskStatus	Pending, running, completed or cancelled
RetryCount	Number of upload attempts
DozeStatus	Active or inactive
CriticalFlag	Indicates whether an event is critical

Runtime State Representation:

STATE = {

SensorData,

NetworkStatus,

BatteryLevel,

PermissionStatus,

EncryptionStatus,

TaskStatus,

RetryCount,

DozeStatus,

CriticalFlag

}

This state helps the application resume waiting tasks, avoid duplicate uploads and safely cancel invalid operations.

A1.3 – Define the Pruning / Feasibility Condition (1 Mark)

Pruning means rejecting or postponing a task when it cannot execute safely or correctly.

Feasibility Conditions:

1. If permission is revoked, cancel the sensor collection task.
2. If internet is unavailable, defer the upload task.
3. If battery is critically low, postpone non-critical uploads.
4. If sensitive data is not encrypted, reject the upload.
5. If the user logs out, cancel user-specific background tasks.
6. If a record is already uploaded, remove it from the pending queue.

Pruning Rule:

IF PermissionStatus = REVOKED

CANCEL task

ELSE IF EncryptionStatus = FALSE

REJECT upload

ELSE IF NetworkStatus = OFFLINE

DEFER upload

ELSE IF BatteryLevel < MinimumThreshold

DEFER non-critical task

ELSE

EXECUTE task

This rule prevents unnecessary processing, battery consumption and security violations.

A1.4 – Design Complete Architectural Pseudocode (3 Marks)

Algorithm: Constraint-Aware Background Task Scheduling

ALGORITHM BackgroundTaskScheduler

INPUT:

SensorEvents

NetworkStatus

BatteryLevel

PermissionStatus

UserStatus

INITIALIZE:

TaskQueue = Empty

CompletedTasks = Empty

WHILE Application is Active

RECEIVE new sensor event

```
IF PermissionStatus = GRANTED
   STORE sensor reading securely
   ADD reading to TaskQueue
ELSE
   REJECT sensor collection
END IF

SELECT next task from TaskQueue

IF UserStatus = LOGGED_OUT
   CANCEL user-specific tasks
   CLEAR temporary sensitive data
   EXIT current scheduling cycle
END IF

IDENTIFY task type

IF task type = SENSOR_COLLECTION
   SELECT foreground sensor operation
   CHECK sensor permission
ELSE
   SELECT WorkManager upload worker
   CHECK network requirement
END IF

CHECK feasibility conditions

IF permission is REVOKED
   CANCEL task
```

```
    RECORD cancellation reason

ELSE IF data is not ENCRYPTED
    REJECT task
    RECORD security failure

ELSE IF task requires network
    AND NetworkStatus = OFFLINE
    DEFER task

ELSE IF BatteryLevel is LOW
    AND task is NON-CRITICAL
    DEFER task

ELSE
    ENQUEUE or START task

    EXECUTE task

    IF execution SUCCESSFUL
        MARK task as COMPLETED
        STORE upload confirmation
        REMOVE task from pending queue

    ELSE
        INCREMENT RetryCount

        IF RetryCount exceeds maximum
            CANCEL task
```

```
            RECORD failure
        ELSE
            RESCHEDULE task
        END IF
    END IF
END IF

IF CriticalFlag = TRUE
    SEND secure notification
END IF

END WHILE

OUTPUT:
    CompletedTasks
    PendingTasks
    CancelledTasks

END ALGORITHM
```

Android Components Used

Component	**Purpose**
Explicit Intent	Starts a specific application component
Implicit Intent	Requests an action from a suitable application component
Ordered Broadcast	Delivers broadcast messages according to receiver priority
WorkManager	Schedules reliable deferred background work
Foreground Service	Supports ongoing user-visible work

Notification Manager	Displays critical health alerts

Explanation:

The scheduler first checks permissions, battery, network and encryption status. If all conditions are satisfied, it executes the task. Otherwise, it defers, rejects or cancels the task. Failed tasks can be retried according to a predefined limit.

A1.5 – Trace the Design (1 Mark)

Consider the following scenario with six events.

Step	Event	Constraint Check	Action	Result
1	Sensor reading received	Permission granted	Collect data	Reading stored
2	Internet disconnected	Network unavailable	Defer upload	Task pending
3	Device enters Doze	Background restriction	Wait for eligible execution	Task deferred
4	Critical health alert	Critical event detected	Send notification	User alerted
5	Internet restored	Network available	Upload encrypted data	Upload successful
6	User logs out	User inactive	Cancel pending user tasks and clear temporary data	Secure termination

Conclusion:

The constraint-aware architecture ensures secure data collection, reliable background execution and efficient resource usage. It also maintains application responsiveness by deferring tasks when the device cannot safely execute them.

TASK A2 – DON'T SYNCHRONISE WHAT CANNOT BE SAFE (5 Marks)

Title: Improve a Weak Data-Replication Design

A2.1 – Diagnose the Computational & Security Waste (1 Mark)

The given algorithm uploads every local record without checking its priority, network condition, battery level or security status.

Problems in the Existing Algorithm:

1. It uploads unnecessary records repeatedly.
2. It does not check whether records are already synchronised.
3. It wastes battery when the network is unstable.
4. It may upload sensitive information without encryption.
5. It does not handle data conflicts.
6. It does not prioritise critical records.
7. It may waste network bandwidth.

Conclusion:

The existing algorithm performs unnecessary operations and creates security risks because it does not validate records before uploading.

A2.2 – Redesign Using Incremental & Constraint-Aware Synchronisation (2 Marks)

Proposed Algorithm: IncrementalSync

ALGORITHM IncrementalSync(LocalStore, RemoteAPI)

INITIALIZE:

SyncQueue = Empty

FOR each record R in LocalStore

IF R is already synchronised

DISCARD R from SyncQueue

ELSE IF R is invalid

DISCARD R

ELSE IF R is not encrypted

REJECT R

```
    ELSE IF RemoteAPI has a newer conflicting version
        RESOLVE CONFLICT
        OR DEFER until resolution is possible

    ELSE IF BatteryLevel is LOW
        DEFER R

    ELSE IF NetworkQuality is POOR
        DEFER R

    ELSE IF R has HIGH priority
        ADD R to front of SyncQueue

    ELSE
        ADD R to SyncQueue
    END IF

END FOR

WHILE SyncQueue is NOT EMPTY

    SELECT highest-priority record R

    IF NetworkStatus = AVAILABLE
        AND EncryptionStatus = TRUE

        UPLOAD R
```

```
        IF Upload SUCCESSFUL
            MARK R as SYNCHRONISED
        ELSE
            DEFER R
        END IF

    ELSE
        DEFER R
    END IF

END WHILE

OUTPUT:
    UploadedRecords
    DeferredRecords
    DiscardedRecords
    ConflictRecords

END ALGORITHM
```

Explanation:

The redesigned algorithm checks every record before synchronisation. High-priority records are processed first, while invalid or already synchronised records are removed. Records that cannot be uploaded safely are deferred or rejected.

A2.3 – Demonstrate Pruning (1 Mark)

Consider five healthcare records.

Record	**Priority**	**Size**	**Sensitivity**	**Encryption**
R1 – Heart rate	High	2 KB	High	Yes
R2 – Temperature	Medium	1 KB	Medium	Yes

R3 – Medical history	High	10 KB	Very High	No
R4 – General activity	Low	5 KB	Low	Yes
R5 – Emergency alert	Critical	1 KB	Very High	Yes

Pruning Execution:

Record	Condition	Action
R1	Network available and encrypted	Upload
R2	Network available and encrypted	Upload
R3	Not encrypted	Reject
R4	Battery low and low priority	Defer
R5	Critical and encrypted	Upload

Early Termination Example:

For R3, the algorithm immediately rejects the upload because encryption is unavailable.

Further upload attempts cannot satisfy the security requirement unless the record is encrypted first. Therefore, the unsafe branch is terminated.

Result:

- Uploaded records: R1, R2 and R5.
- Deferred records: R4.
- Rejected records: R3.

A2.4 – Challenge Your Pruning Rule (1 Mark)

If offline-first operation with eventual consistency is required, the pruning condition must be modified.

Reason:

In offline-first applications, users must be able to create and modify records without internet connectivity.

The original algorithm defers uploads when the network is unavailable. However, it must still allow local changes to be stored safely.

Modified Conditions:

1. Store new records locally when offline.
2. Maintain a pending synchronisation queue.
3. Assign unique IDs and version numbers.
4. Synchronise when connectivity becomes available.
5. Resolve conflicts using timestamps or version numbers.
6. Never discard valid offline records merely because the network is unavailable.

Conclusion:

The original pruning rule is not completely valid under offline-first operation because it assumes that network availability is required for progress. Eventual consistency allows local updates first and cloud synchronisation later.

TASK A3 – DISCOVER THE MOBILE-BACKGROUND DESIGN PATTERN (5 Marks)

Comparison Table

Design Component	**Sensor Collection**	**Data Sync**	**Push Notification**
State	Sensor readings and permission status	Pending records and network status	Notification content and delivery status
Decision variable	Whether to collect data	Whether to upload records	Whether to deliver notification
Candidate choices	Collect, pause, cancel	Upload, defer, resolve conflict	Send, retry, cancel
Feasibility / Constraint test	Permission, battery and sensor availability	Network, encryption and battery	Notification permission and valid event
Goal condition	Valid readings stored	Records successfully synchronised	Notification delivered

Pruning / Deferral condition	Permission denied or battery low	Network unavailable or unsafe data	Invalid notification or permission denied
Undo / Cancel operation	Stop collection and release sensor	Cancel upload and restore pending status	Cancel notification and clear temporary data

A3.1 – Identify the Common Pattern (2 Marks)

The three mobile background problems have a common computational and architectural structure.

Common Characteristics:

1. All three processes operate using background tasks.
2. Each process maintains a runtime state.
3. Each process has multiple execution choices.
4. Every task must satisfy feasibility conditions.
5. Unnecessary or unsafe operations can be pruned.
6. Failed operations may require retry or cancellation.
7. Each process has a specific goal condition.

Common Computational Pattern:

Initialize Runtime State

Select Background Task

Check Feasibility Constraints

Feasible

Execute task

Not feasible

Defer / Cancel

Update State and Handle Result

Conclusion:

Sensor collection, data synchronisation and push notification delivery all follow a constraint-aware background scheduling pattern.

A3.2 – Design a Generic Background-Work Framework (2 Marks)

Algorithm: SCHEDULE_OR_EXECUTE(State)

ALGORITHM SCHEDULE_OR_EXECUTE(State)

INITIALIZE:

TaskQueue

CompletedTasks

CancelledTasks

WHILE TaskQueue is NOT EMPTY

SELECT next task T

READ current runtime state

IDENTIFY task type

GENERATE candidate actions

CHECK feasibility conditions

IF task is feasible

EXECUTE task T

IF execution SUCCESSFUL

UPDATE State

MARK T as COMPLETED

ELSE

```
            RESTORE previous state
            INCREMENT retry count

            IF retry limit exceeded
               CANCEL T
            ELSE
               RESCHEDULE T
            END IF
         END IF

      ELSE IF task can be executed later

         DEFER T
         ADD T to pending queue

      ELSE

         CANCEL T
         RECORD cancellation reason

      END IF

   END WHILE

   RETURN:
      CompletedTasks
      CancelledTasks
      UpdatedState
```

END ALGORITHM

Explanation:

This framework can be used for sensor collection, offline data synchronisation and push notification delivery. It selects tasks, checks constraints, executes valid operations and handles failures.

A3.3 – Analyse the Role of Undo / Cancellation (1 Mark)

Undo and cancellation are important because background tasks may fail or become invalid while executing.

Example: Data Synchronisation

Suppose an application starts uploading a medical record. During the upload, the user logs out.

If cancellation is not handled properly:

- The upload may continue after logout.
- Sensitive information may be exposed.
- Temporary files may remain in storage.
- The application may maintain an incorrect task status.

Required Undo Operations:

1. Stop the current upload.
2. Cancel pending user-specific tasks.
3. Remove temporary sensitive data.
4. Restore the record to a safe pending state when appropriate.
5. Update the task status.

Conclusion:

Undo and cancellation maintain data consistency, privacy and system reliability. They prevent incomplete or unsafe background operations from affecting the application.

TASK A4 – TRACTABILITY: LOCAL VS CLOUD VS HYBRID PROCESSING (4 Marks)

Problem Analysis Table

Problem	**What constitutes a solution?**	**Efficiently verified?**	**Complexity / Resource Class**	**Reason**

1. On-device image classification	Correctly classify an image using a tiny model	Yes, using test images and accuracy measurements	On-device feasible	Small models can execute directly on mobile devices
2. Cloud-based speech recognition	Convert speech into accurate text using cloud processing	Yes, using sample audio and recognition accuracy	Network-bound	Requires internet connectivity and server processing
3. Peer-to-peer file sharing	Transfer files correctly between nearby devices	Yes, by checking file integrity and transfer completion	Battery-critical	Continuous communication consumes battery and network resources
4. End-to-end encrypted messaging	Securely send and receive encrypted messages	Yes, using encryption and decryption tests	Security-critical	Confidentiality and key management are important
5. Real-time multiplayer game synchronisation	Maintain consistent game state between players	Yes, using latency and synchronisation tests	Hybrid required	Requires local processing and network communication
6. Large-scale offline map rendering	Render map tiles correctly without internet	Yes, by testing rendering accuracy and performance	On-device feasible	Local vector-tile processing supports offline usage

Inquiry 1

Why is "runs slowly on a phone" not an adequate definition of a performance problem in mobile computing?

Running slowly is only one aspect of mobile application performance.

Mobile applications also depend on:

- CPU processing capacity.

- Memory availability.
- Battery consumption.
- Network connectivity.
- Data transfer speed.
- Rendering and frame rate.
- Security and privacy requirements.

For example, a cloud-based application may execute computation quickly on a server but experience delays because of poor internet connectivity.

Therefore, mobile performance must be evaluated using execution time, memory usage, battery consumption, network latency and user experience.

Inquiry 2

Why is the distinction between local processing and cloud offloading important when interpreting scalability and security of a mobile application?

Local processing executes tasks directly on the mobile device, whereas cloud offloading transfers tasks to remote servers.

Factor	**Local Processing**	**Cloud Processing**
Execution	On mobile device	Remote server
Internet	Not always required	Usually required
Battery	Uses device resources	Uses network and device resources
Scalability	Limited by device capacity	Can use scalable server resources
Security	Data remains locally	Data is transmitted over a network
Latency	Depends on device performance	Depends on network and server response

Conclusion:

Local processing is useful for offline operations and privacy-sensitive tasks. Cloud processing is useful for computationally expensive operations and large-scale workloads. Hybrid processing combines both approaches to achieve better performance, security and scalability.

TASK A5 – DESIGN WHEN EXACT OPTIMISATION MAY BE TOO EXPENSIVE (4 Marks)

Title: Graphics / UI Performance and Approximation

A5.1 – Exact Perspective (1 Mark)

An exact combinatorial search examines all possible combinations of decoding resolution, cache size and prefetch distance to identify the configuration with minimum energy consumption while maintaining smooth scrolling.

Example:

Assume the application has:

- 3 decoding resolution levels.
- 4 cache-size options.
- 3 prefetch-distance options.

Total configurations:

$$3 \times 4 \times 3 = 36$$

The exact algorithm evaluates all 36 configurations and selects the one with the minimum cost.

Cost Function:

$$Cost = \alpha(FD) + \beta(EC) + \gamma(MP)$$

Where:

- FD = Frame drops.
- EC = Energy consumption.
- MP = Memory pressure.
- α, β, γ= Weight factors.

Limitations:

1. The number of configurations increases as parameters increase.
2. More feed items increase testing requirements.
3. Different devices have different hardware capabilities.
4. Exhaustive evaluation consumes additional processing time.
5. It is difficult to perform exact optimisation continuously.

Conclusion:

Exact optimisation can find the best configuration for a small search space, but it becomes expensive for large mobile applications.

A5.2 – Approximation / Heuristic Design (2 Marks)

Algorithm: AdaptiveFeedOptimizer

```
ALGORITHM AdaptiveFeedOptimizer

INITIALIZE:
    Quality = HIGH
    CacheSize = 5
    PrefetchDistance = 3

    FrameTime = MeasureFrameTime()
    BatteryLevel = MeasureBattery()
    MemoryUsage = MeasureMemory()

REPEAT

    UPDATE current performance metrics

    IF FrameTime > 16 ms

        REDUCE decoding quality

        IF MemoryUsage is HIGH
            REDUCE CacheSize
        END IF

        REDUCE PrefetchDistance
```

```
    ELSE IF BatteryLevel is LOW

        REDUCE PrefetchDistance
        REDUCE decoding quality
    ELSE IF FrameTime < 12 ms
        AND BatteryLevel is HIGH
        INCREASE decoding quality
        INCREASE PrefetchDistance
    END IF

    APPLY selected parameters

    UPDATE:
        FrameTime
        BatteryLevel
        MemoryUsage
UNTIL FrameTime <= 16 ms
    AND parameters remain stable
OUTPUT:
    Quality
    CacheSize
    PrefetchDistance
    PerformanceMetrics
END ALGORITHM
```

Explanation:

The adaptive algorithm measures frame time, battery and memory usage. If the frame time exceeds 16 ms, it reduces decoding quality and prefetching. If sufficient resources are available, it gradually increases quality.

This approach avoids evaluating every possible configuration.

A5.3 – What Have We Sacrificed? (1 Mark)

There is an important difference between finding a feasible solution and proving an optimal solution.

1. Finding a Feasible, Smooth Scrolling Experience

- Maintains frame time below 16 ms.
- Reduces visible frame drops.
- Controls memory consumption.
- Reduces unnecessary background processing.
- Provides a responsive user experience.

2. Proving Minimum Energy Consumption

- Requires comparing possible configurations.
- Needs a well-defined energy cost function.
- Requires evaluating different device conditions.
- May require exhaustive search or a suitable exact optimisation method.

Comparison:

Factor	**Approximate Solution**	**Exact Solution**
Search	Limited	All configurations
Execution time	Usually lower	Can be higher
Energy optimisation	Not guaranteed	Optimal for the defined model
Solution quality	Near-optimal or feasible	Optimal
Scalability	Better for large search spaces	Can become expensive

Conclusion:

The approximate method provides a practical and responsive scrolling experience with lower computational overhead. However, it cannot guarantee minimum energy consumption. It sacrifices the proof of optimality in exchange for faster decision-making.

TASK B1 – IMPLEMENT AND INSTRUMENT YOUR BACKGROUND ARCHITECTURE (7 Marks)

Title: Android Intents / Services / WorkManager

Aim

To implement a constraint-aware background task scheduling system that collects sensor readings, checks network and battery conditions, securely uploads data and records execution statistics.

B1 – Implementation

System Requirements:

1. Maintain a queue of background tasks.
2. Check battery and network conditions.
3. Check permission and encryption status.
4. Execute eligible tasks.
5. Defer or cancel unsuitable tasks.
6. Record execution statistics.
7. Compare FIFO and priority-based scheduling.

Algorithm

1. Start the application.
2. Create a background task queue.
3. Receive sensor readings.
4. Check user permission.
5. Check encryption status.
6. Check battery and network conditions.
7. Execute eligible tasks.
8. Defer or cancel unsuitable tasks.
9. Record the results.
10. Display the final statistics.

Python Program – Instrumented Background Architecture

```
# TASK B1 - Background Work Architecture
```

```
class BackgroundScheduler:

    def __init__(self):
        self.enqueued = 0
        self.deferred = 0
        self.cancelled = 0
        self.successful = 0
        self.max_workers = 0
        self.uploaded = []

    def execute(self, tasks, battery, network,
                permission, doze, strategy):

        self.enqueued = len(tasks)

        # FIFO or Priority scheduling
        if strategy == "PRIORITY":
            tasks = sorted(
                tasks,
                key=lambda x: (
                    -x["priority"],
                    x["size"]
                )
            )

        active_workers = 0

        for task in tasks:
```

```
print("\nProcessing:", task["name"])


# Permission check
if permission == "NO":
```

```
Processing: Heart Rate
Upload successful: Heart Rate

Processing: Temperature
Upload successful: Temperature

Processing: Medical History
Cancelled: Data not encrypted

Processing: Activity
Upload successful: Activity

Processing: Emergency Alert
Upload successful: Emergency Alert

----- FINAL STATISTICS -----
Work items enqueued: 5
Deferred tasks: 0
Cancelled tasks: 1
Successful uploads: 4
Maximum concurrent workers: 1
Uploaded records: ['Heart Rate', 'Temperature', 'Activity', 'Emergency Alert']
Residual backlog: 0
```

Result:

The background scheduling system successfully executes eligible tasks, rejects unencrypted records and records the number of successful, deferred and cancelled operations.

B1.1 – Validate the Implementation (2 Marks)

The implementation is tested using three different scenarios.

Test Case 1 – Good Connectivity and Battery

Parameter	**Value**
Battery	80%
Network	Online
Permission	Granted
Doze Mode	Disabled

Encryption	Enabled for valid records

Expected behaviour: Eligible records should be uploaded successfully.

Test Case 2 – Doze Mode and Low Battery

Parameter	**Value**
Battery	10%
Network	Online
Permission	Granted
Doze Mode	Enabled

Expected behaviour: Non-critical tasks should be deferred, while eligible critical tasks can execute.

Test Case 3 – Permission Revoked During Execution

Parameter	**Value**
Battery	70%
Network	Online
Permission	Revoked
Doze Mode	Disabled

Expected behaviour: The current sensor operation should stop safely, and pending sensor tasks should be cancelled.

Test Analysis

Test	**Purpose**	**Expected Result**
Test 1	Verify normal execution	Successful uploads
Test 2	Verify resource constraints	Non-critical tasks deferred

Test 3	Verify permission handling	Tasks cancelled safely

B1.2 – Compare Design with Implementation (1 Mark)

Part A Design	**Part B Implementation**
Select next task	Task queue
Check permission	Permission condition
Check encryption	Encryption condition
Check battery	Battery threshold
Check network	Network status
Execute task	Upload operation
Handle failure	Defer or cancel
Update state	Statistics

Modification Made:

In the implementation, a numerical priority value is used to select tasks. A simple simulation is used to represent network, battery and Doze conditions.

This modification makes it easier to test multiple scheduling conditions and compare their results.

B1.3 – Improve the Scheduling Order (2 Marks)

Two scheduling approaches are implemented.

Version 1 – FIFO Scheduling

- Processes tasks in arrival order.
- Simple to implement.
- Does not consider task importance.
- Critical tasks may wait behind ordinary tasks.

Version 2 – Priority-Based Scheduling

The following priority levels are used:

Task	Priority
Emergency Alert	4
Heart Rate	3
Temperature	2
Activity	1
Medical History	1

Selection Rule:

$$PriorityScore = TaskPriority$$

Tasks with higher priority are selected first. If priorities are equal, smaller tasks are selected first.

Python Code – Scheduling Comparison

```
# B1.3 - Compare FIFO and Priority Scheduling

import copy

print("VERSION 1 - FIFO SCHEDULING")

fifo = BackgroundScheduler()

fifo.execute(
    copy.deepcopy(tasks),
    battery=80,
    network="ONLINE",
    permission="YES",
    doze="NO",
```

```
    strategy="FIFO"
)

print("\nVERSION 2 - PRIORITY SCHEDULING")

priority = BackgroundScheduler()

priority.execute(
    copy.deepcopy(tasks),
    battery=80,
    network="ONLINE",
    permission="YES",
    doze="NO",
    strategy="PRIORITY"
)
```

Expected Scheduling Order:

Version	Processing Order
FIFO	Heart Rate → Temperature → Medical History → Activity → Emergency Alert
Priority	Emergency Alert → Heart Rate → Temperature → Activity → Medical History

B1.4 – Measure the Effect (2 Marks)

Experimental Comparison

The following results are obtained for the same five records under identical conditions.

Measure	Version 1 – FIFO	Version 2 – Priority
Work items enqueued	5	5
Deferred / cancelled	1	1

Successful executions	4	4
Peak concurrent workers	1	1
Emergency Alert position	5	1

Analysis:

The priority scheduling approach processes emergency information earlier than FIFO scheduling.

Both approaches perform the same number of successful uploads under these conditions. However, priority scheduling improves the delivery order of critical information.

Conclusion:

Priority scheduling is more suitable for healthcare applications because critical information can be processed before ordinary records. It does not necessarily reduce the total number of failed operations, but it improves the importance-based execution order.

TASK B2 – ENGINEER AN ADVERSARIAL WORKLOAD (5 Marks)

Title: Same Size, Very Different Behaviour

Aim

To construct two workloads containing the same number of records but having different network, battery and security conditions, and compare their execution behaviour.

B2 – Workload Design

Two workloads are created with five records each.

Workload A – Easy Workload

- Good network connectivity.
- High battery level.
- All records encrypted.
- All permissions granted.
- Mostly normal-priority records.

Workload B – Difficult Workload

- Poor network connectivity.
- Low battery level.
- Doze mode enabled.

- One record is not encrypted.
- Permission may be revoked.

Sample Input

Workload A

Record	Priority	Size	Encryption
A1	2	2 KB	Yes
A2	2	3 KB	Yes
A3	3	1 KB	Yes
A4	1	2 KB	Yes
A5	2	1 KB	Yes

Workload B

Record	Priority	Size	Encryption
B1	2	2 KB	Yes
B2	1	3 KB	Yes
B3	3	1 KB	No
B4	1	2 KB	Yes
B5	2	1 KB	Yes

Before Running – Prediction

Parameter	Workload A	Workload B
Number of records	5	5
Battery	90%	10%

Network	Online	Offline
Doze mode	Disabled	Enabled
Permission	Granted	Granted
Encryption	All encrypted	One unencrypted
Expected deferrals	Low	High
Expected cancellations	Low	Higher

Prediction:

Workload B will require more unsuccessful attempts and produce more deferred or cancelled tasks because it contains network restrictions, low battery and an unencrypted record.

B2 – Python Implementation

```
# TASK B2 - Adversarial Workload Testing

workload_A = [
    {"name": "A1", "priority": 2, "size": 2,
     "encrypted": "YES"},
    {"name": "A2", "priority": 2, "size": 3,
     "encrypted": "YES"},
    {"name": "A3", "priority": 3, "size": 1,
     "encrypted": "YES"},
    {"name": "A4", "priority": 1, "size": 2,
     "encrypted": "YES"},
    {"name": "A5", "priority": 2, "size": 1,
     "encrypted": "YES"}
]

workload_B = [
    {"name": "B1", "priority": 2, "size": 2,
     "encrypted": "YES"},
```

```
    {"name": "B2", "priority": 1, "size": 3,
     "encrypted": "YES"},
    {"name": "B3", "priority": 3, "size": 1,
     "encrypted": "NO"},
    {"name": "B4", "priority": 1, "size": 2,
     "encrypted": "YES"},
    {"name": "B5", "priority": 2, "size": 1,
     "encrypted": "YES"}
]
print("WORKLOAD A - EASY")
test_A = BackgroundScheduler()
test_A.execute(
    workload_A,
    battery=90,
```

```
    VERSION 1 - FIFO SCHEDULING

••• Processing: Heart Rate
    Upload successful: Heart Rate

    Processing: Temperature
    Upload successful: Temperature

    Processing: Medical History
    Cancelled: Data not encrypted

    Processing: Activity
    Upload successful: Activity

    Processing: Emergency Alert
    Upload successful: Emergency Alert

    ----- FINAL STATISTICS -----
    Work items enqueued: 5
    Deferred tasks: 0
    Cancelled tasks: 1
    Successful uploads: 4
    Maximum concurrent workers: 1
    Uploaded records: ['Heart Rate', 'Temperature', 'Activity', 'Emergency Alert']
    Residual backlog: 0
```

```
VERSION 2 - PRIORITY SCHEDULING

Processing: Emergency Alert
Upload successful: Emergency Alert

Processing: Heart Rate
Upload successful: Heart Rate

Processing: Temperature
Upload successful: Temperature

Processing: Activity
Upload successful: Activity

Processing: Medical History
Cancelled: Data not encrypted

----- FINAL STATISTICS -----
Work items enqueued: 5
Deferred tasks: 0
Cancelled tasks: 1
Successful uploads: 4
Maximum concurrent workers: 1
Uploaded records: ['Emergency Alert', 'Heart Rate', 'Temperature', 'Activity']
Residual backlog: 0
```

Experimental Comparison

Measurement	Workload A	Workload B
Records	5	5
Processed	5	5
Deferred	0	4
Cancelled	0	1
Successful uploads	5	0
Residual backlog	0	4

Here, processed means records examined by the scheduler, not successfully uploaded records.

Investigation

Workload A executes successfully because all the required conditions are satisfied.

Workload B experiences more deferrals because:

1. Internet connectivity is unavailable.
2. Battery level is below the minimum threshold.

3. Doze mode restricts ordinary background operations.
4. One record is not encrypted and must be rejected.

Structural Property:

The difficult workload contains more constraint violations. Therefore, the algorithm spends more effort checking conditions and managing deferred tasks.

Conclusion:

Input size alone cannot describe the computational effort of a mobile background application. Two workloads with the same number of records can have different execution costs because of network conditions, battery constraints, security requirements and task priorities.

TASK B3 – EXACT VS APPROXIMATE UI / GRAPHICS OPTIMISATION (8 Marks)

Title: Scrolling Feed Performance Optimisation

Aim

To compare an exact exhaustive search algorithm and an approximate adaptive algorithm for optimising a mobile scrolling feed.

Problem Description

A social media application displays images and videos continuously. The application must maintain a frame time below 16 milliseconds while reducing energy consumption and memory pressure.

Two solutions are developed:

- Solution A – Exact / Exhaustive Search.
- Solution B – Approximate / Adaptive Search.

B3 – Define the Cost Function

The cost function considers frame drops and energy consumption.

$$Cost = 10(FD) + EC$$

Where:

- FD = Number of frame drops.
- EC = Estimated energy consumption.

A lower cost represents a better configuration.

Solution A – Exact / Exhaustive Search

The exhaustive algorithm examines all possible configurations.

Parameters:

Parameter	Available Options
Decoding quality	Low, Medium, High
Cache size	1, 2, 3
Prefetch distance	1, 2, 3

Total configurations:

$$3 \times 3 \times 3 = 27$$

The algorithm calculates the cost of every configuration and selects the minimum-cost configuration.

Solution B – Approximate / Adaptive Search

The adaptive algorithm begins with a default configuration and adjusts the parameters according to frame time and resource consumption.

Selection Rules:

1. If frame time exceeds 16 ms, reduce decoding quality.
2. If memory usage is high, reduce cache size.
3. If battery is low, reduce prefetch distance.
4. If performance is good, gradually increase quality.
5. Stop when the system reaches a stable feasible configuration.

Python Implementation – Exact and Approximate Algorithms

```
# TASK B3 - Exact vs Approximate UI Optimization

from itertools import product

# Three quality levels
qualities = [1, 2, 3]
```

```
# Cache sizes
caches = [1, 2, 3]

# Prefetch distances
prefetches = [1, 2, 3]



# Cost calculation function
def calculate_cost(q, c, p, workload):

    frame_time = (
        workload["base"]
        + 5 * q
        - 2 * c
        - 2 * p
    )

    energy = (
        5 * q + 3 * c + 2 * p
    )

    frame_drops = max(
        0,
        frame_time - 16
    )

    cost = 10 * frame_drops + energy
```

```
    WORKLOAD A - EASY
...
    Processing: A1
    Upload successful: A1

    Processing: A2
    Upload successful: A2

    Processing: A3
    Upload successful: A3

    Processing: A4
    Upload successful: A4

    Processing: A5
    Upload successful: A5

    ----- FINAL STATISTICS -----
    Work items enqueued: 5
    Deferred tasks: 0
    Cancelled tasks: 0
    Successful uploads: 5
    Maximum concurrent workers: 1
    Uploaded records: ['A1', 'A2', 'A3', 'A4', 'A5']
    Residual backlog: 0
```

```
  WORKLOAD B - DIFFICULT

  Processing: B1
  Deferred: No network

  Processing: B2
  Deferred: No network

  Processing: B3
  Cancelled: Data not encrypted

  Processing: B4
  Deferred: No network

  Processing: B5
  Deferred: No network

  ----- FINAL STATISTICS -----
  Work items enqueued: 5
  Deferred tasks: 4
  Cancelled tasks: 1
  Successful uploads: 0
  Maximum concurrent workers: 0
  Uploaded records: []
  Residual backlog: 4
```

B3.1 – Correctness on Small Instances (2 Marks)

Three test instances are created with different workload conditions.

Expected Comparison:

Instance	Base Frame Time	Exact Configurations Evaluated	Adaptive Configurations

Instance 1	10 ms	27	Limited
Instance 2	15 ms	27	Limited
Instance 3	20 ms	27	Limited

The exact algorithm evaluates all 27 configurations for each instance.

The adaptive algorithm evaluates only the configurations encountered during its adjustment process.

Correctness Analysis:

The exact algorithm finds the minimum cost among the defined configurations. The adaptive algorithm finds a configuration based on its selection rules.

The results may match in some cases and differ in other cases. This difference is expected because the adaptive algorithm does not examine the complete search space.

B3.2 – Find a Counter-example (2 Marks)

A counter-example demonstrates that the adaptive algorithm may not find the minimum-cost configuration.

Consider the following workload:

$$BaseFrameTime = 20$$

Example Configuration Comparison:

Quality	Cache	Prefetch	Frame Time	Energy	Cost
1	1	1	19 ms	10	40
1	3	3	15 ms	20	20
2	3	3	20 ms	25	65
3	3	3	25 ms	30	120

Cost is calculated using:

$$Cost = 10(FD) + EC$$

For example, for quality 1, cache 1 and prefetch 1:

$$Cost = 10(19 - 16) + 10 = 40$$

Adaptive Decision:

The adaptive algorithm begins with high quality, large cache and high prefetch distance.

It reduces quality first. It may then reduce cache based on its energy threshold.

However, a configuration with lower quality and a larger cache may provide a lower total cost.

Reason for Failure:

The adaptive algorithm makes local decisions based on current measurements. It does not evaluate all possible combinations.

Therefore, it may select a feasible but suboptimal configuration.

B3.3 – Increase the Problem Size (2 Marks)

The problem size is increased by adding more feed items and quality levels.

Experimental Configuration:

Experiment	Feed Items	Quality Levels	Cache Options	Prefetch Options	Total Configurations
Test 1	10	3	3	3	27
Test 2	50	5	4	4	80
Test 3	100	8	5	5	200

The number of configurations is calculated as:

$$Total = Q \times C \times P$$

Python Code – Scalability Measurement

```
# B3.3 - Scalability Measurement

tests = [
```

```
    (10, 3, 3, 3),
    (50, 5, 4, 4),
    (100, 8, 5, 5),
    (500, 10, 8, 8)
]

print("SCALABILITY ANALYSIS")

for items, q, c, p in tests:

    exact_operations = q * c * p

    adaptive_operations = min(
        15,
        q + c + p
    )

    print("\nFeed items:", items)
    print("Quality levels:", q)
    print("Cache options:", c)
    print("Prefetch options:", p)

    print(
        "Exact configurations:",
        exact_operations
    )

    print(
        "Adaptive operation estimate:",
```

```
    adaptive_operations
)
```

```
 Instance 1
Exact configuration: (1, 1, 1)
Exact cost: 10
Configurations evaluated: 27
Adaptive configuration: (2, 1, 3)
Adaptive cost: 19
Parameter changes: 1
Cache evictions: 2
Result: Costs do not match

 Instance 2
Exact configuration: (1, 1, 1)
Exact cost: 10
Configurations evaluated: 27
Adaptive configuration: (2, 3, 3)
Adaptive cost: 25
Parameter changes: 1
Cache evictions: 0
Result: Costs do not match

 Instance 3
Exact configuration: (1, 2, 3)
Exact cost: 17
Configurations evaluated: 27
Adaptive configuration: (1, 3, 3)
Adaptive cost: 20
Parameter changes: 2
Cache evictions: 0
Result: Costs do not match
```

Analysis:

The exact algorithm evaluates every possible configuration. As the number of quality, cache and prefetch options increases, the search space becomes larger.

The adaptive algorithm uses a limited number of adjustment steps. Therefore, it can reduce the number of configuration evaluations.

B3.4 – Engineering Trade-off (2 Marks)

Comparison of Exact and Approximate Methods

Factor	**Exact Method**	**Approximate Method**
Search strategy	Exhaustive	Greedy / Adaptive
Configuration evaluation	All combinations	Selected combinations
Execution cost	Higher	Lower

Optimality	Guaranteed within the defined search space	Not guaranteed
Scalability	Limited for large search spaces	Better
Decision speed	Slower	Faster

What did the approximate solution gain?

- Reduced configuration evaluations.
- Faster parameter selection.
- Lower computational overhead.
- Better suitability for continuous adaptation.
- Faster response to changing device conditions.

What did it sacrifice?

- Guaranteed minimum cost.
- Complete exploration of all possible configurations.
- Proof that the selected configuration is optimal.

Realistic Application:

A social-media application with thousands of users and different mobile devices can use an adaptive approach. It can adjust image quality, cache size and prefetch distance based on real-time performance measurements.

Conclusion:

The approximate solution is suitable for production mobile applications where quick decisions and responsiveness are more important than proving global optimality.

TASK B4 – FINAL ENGINEERING DECISION (5 Marks)

Title: Multimedia Framework Selection under Production Constraints

Aim

To design a practical heuristic for selecting video resolution, bitrate and upload timing for mobile devices with different hardware, battery levels and network conditions.

Problem Description

A production mobile application must capture, encode and upload short video clips from thousands of users.

Different mobile devices have different:

- Camera capabilities.
- CPU processing power.
- Battery capacity.
- Network bandwidth.
- Available memory.

Therefore, the application must select suitable encoding parameters without performing expensive exhaustive searches.

B4.1 – Implement a Practical Strategy (2 Marks)

Proposed Selection Rules

Condition	Resolution	Bitrate	Upload Timing
Battery above 60%, network above 5 Mbps	1080p	4 Mbps	Upload immediately
Battery 20–60%, network 2–5 Mbps	720p	2 Mbps	Upload when stable
Battery below 20% or network below 2 Mbps	480p	1 Mbps	Defer if necessary

Algorithm

1. Read device battery level.
2. Read network bandwidth.
3. Read available CPU capacity.
4. Select resolution based on device conditions.
5. Select suitable bitrate.
6. Estimate encoding time.
7. Estimate upload time.
8. Calculate energy consumption.
9. Calculate remaining battery impact.
10. Display the selected configuration.

Python Implementation – Adaptive Video Encoding

TASK B4 - Multimedia Encoding Strategy

```
devices = [
    {
        "name": "Device A",
        "battery": 90,
        "network": 10,
        "cpu": 90
    },
    {
        "name": "Device B",
        "battery": 45,
        "network": 4,
        "cpu": 60
    },
    {
        "name": "Device C",
        "battery": 15,
        "network": 1,
        "cpu": 30
    },
    {
        "name": "Device D",
        "battery": 75,
        "network": 3,
        "cpu": 70
    }
]
```

Approximate video clip size in MB

clip_size = 20

Battery capacity in mAh

battery_capacity = 4000

Expected Output

Illustrative estimates from the stated simplified model.

Device	Resolution	Bitrate	Encoding Time	Upload Time
Device A	1080p	4 Mbps	8 sec	16 sec
Device B	720p	2 Mbps	10 sec	40 sec
Device C	480p	1 Mbps	15 sec	0 sec
Device D	720p	2 Mbps	10 sec	53.33 sec

Output Interpretation:

- Device A has a good battery and network, so it uses high-quality encoding.
- Device B uses medium-quality encoding to balance performance and energy.
- Device C has a low battery and weak network, so it uses low-quality encoding and defers uploading.
- Device D uses medium-quality encoding because network bandwidth is limited.

B4.2 – Challenge Your Strategy (1 Mark)

Counter-example:

Consider a device with the following conditions:

Parameter	Value
Battery	55%

Network bandwidth	6 Mbps
CPU capacity	80%
Video size	20 MB

The heuristic selects 720p because the battery is below 60%.

However, the device has good network bandwidth and CPU capacity. A 1080p configuration may provide better video quality while still meeting the required performance constraints.

Why the Heuristic May Fail:

1. It uses fixed thresholds.
2. It does not examine every possible resolution and bitrate.
3. It does not consider the user's video-quality preference.
4. It uses estimated rather than measured energy consumption.
5. It may select a lower-quality configuration unnecessarily.

Conclusion:

The heuristic is fast and practical, but it does not guarantee an optimal encoding configuration for every device.

B4.3 – Production Decision Note (2 Marks)

Algorithm / Architecture Engineering Decision Note

1. Why is exhaustive search unattractive at production scale?

Exhaustive search evaluates every combination of resolution, bitrate, upload time and device conditions.

When thousands of users upload videos simultaneously, evaluating all possible combinations increases computational overhead and decision-making time.

Therefore, exhaustive search is not suitable for every real-time encoding decision.

2. What guarantee does the chosen strategy provide?

The heuristic provides a practical selection based on predefined battery, network and CPU thresholds.

It does not guarantee a globally optimal configuration. However, it provides a fast decision within the defined rules.

3. What computational and energy advantages does it provide?

- Reduces unnecessary configuration evaluations.
- Selects encoding parameters quickly.
- Reduces battery consumption under resource constraints.
- Avoids unnecessary high-resolution encoding.
- Defers uploads when network conditions are poor.

4. When can an exact or cloud-assisted approach be appropriate?

An exact approach can be used when the number of encoding configurations is small and sufficient computation time is available.

Cloud-assisted processing can be used for:

- Large-scale video processing.
- Advanced video compression.
- Quality analysis.
- Expensive machine-learning operations.

However, cloud processing requires network connectivity and introduces data-transfer and privacy considerations.

5. What evidence should be monitored before deployment?

Metric	**Purpose**
Encoding time	Measure processing speed
Upload time	Measure network performance
Battery consumption	Measure energy efficiency
Video quality	Check output quality
Upload failure rate	Measure reliability
Memory usage	Monitor device resources
Network bandwidth	Evaluate connectivity
User experience	Evaluate application usability

Final Engineering Decision:

A hybrid adaptive encoding strategy is recommended for production mobile applications. The mobile device selects resolution and bitrate based on local battery, CPU and network conditions. Cloud processing can be used for computationally expensive operations when connectivity is available.

This strategy provides a balance between video quality, processing speed, battery efficiency and scalability.

PART B – Important Experimental Observations

Overall Results

Task	Main Investigation	Expected Outcome
B1	Background scheduling	Priority scheduling delivers critical records earlier
B2	Adversarial workloads	Difficult workload produces more deferrals
B3	Exact vs adaptive optimisation	Exact search guarantees minimum cost within the search space
B4	Video encoding strategy	Adaptive encoding balances quality and resources